

HW3 - Containerize and Deploy Your HW2 (now called HW3) App to Cloud Run

Overview

HW2 worked, but only on your laptop: `fastapi run`, `streamlit run`, a `.env` file, and a gcp service account key file in your computer. In HW3 you package that same application into **two Docker images** and deploy them as **two Cloud Run services**, with the service account key delivered by **Secret Manager** and a daily search and save triggered by **Cloud Scheduler**.

You do not write any Python in this homework. All the `.py` files are provided, already complete — they are same as the HW2 solution. Your work is two `Dockerfiles` you write from scratch, plus two shell scripts where you fill in the blanks. (Note - You should not submit your `config.sh`)

Learning Objectives

- Write a Dockerfile for a Python API server and web app.
- Understand why a container must listen on `0.0.0.0`, not `localhost`.
- Deploy to Cloud Run from source, pass configuration in as environment variables, and mount a secret instead of storing it in an image.
- Schedule a recurring job against a deployed service.

Prerequisites

1. **Docker Desktop** running locally.
2. Watch and follow instructions to install and understand how Cloud Run, Cloud Scheduler and gcloud works.
3. The **gcloud CLI**, authenticated:

```
gcloud auth login
gcloud config set project YOUR_PROJECT_ID
```

4. These APIs enabled in your project (the deployment script enables Secret Manager for you. But enable the rest once, by your own):

```
gcloud services enable run.googleapis.com cloudbuild.googleapis.com \
cloudscheduler.googleapis.com
```

Provided Files

Do not change the following

File	What it is
------	------------

File	What it is
<code>fastapi/extract_save_data.py</code>	The FastAPI server from HW2 — POST <code>/search_and_save/jobs</code>
<code>fastapi/user_definition.py</code>	Config read from environment variables
<code>fastapi/requirements.txt</code>	Pinned dependencies for the API image
<code>streamlit/hw3.py</code>	The dashboard from HW2, reading from GCS
<code>streamlit/user_definition.py</code>	Config read from environment variables
<code>streamlit/requirements.txt</code>	Pinned dependencies for the dashboard image
<code>config.sh</code>	Shared settings — has blanks you fill in
<code>gcloud_command.sh</code>	Deployment script — has blanks you fill in
<code>test_hw3.sh</code>	Self-check script — run it, don't edit it

Files You Must Create / Complete

1. `fastapi/Dockerfile` (create from scratch)

Build an image that runs the HW3 API server. It needs to:

1. Start with a Python 3.13 base image. Use a `-slim` variant — the full image is roughly ten times larger.
2. Set a working directory
3. Copy all the files to the image's working directory.
4. Install all required python packages and libraries.
5. Expose a port.
6. Run your FastAPI application on 0.0.0.0.

DO NOT copy `.env` or your service account key into the image. Both are supplied at runtime — that is what Step 1 of the deployment script is for.

2. `streamlit/Dockerfile` (create from scratch)

Same as above. **Ensure to use 8501 as a port and 0.0.0.0 as an address.**

The two reference pages that will helpl this configuration:

- [streamlit run CLI reference](#) — shows that any configuration option can be passed on the command line as `--<section>.<option>=<value>`.
- [config.toml reference → Server](#) — lists the two options you need, `server.port` and `server.address`.

3. `config.sh` (fill in **TODO 1–TODO 5**)

PLEASE DO NOT SUBMIT. This helps to set deployment settings including your GCP project id, region, the absolute path to `.env`, the absolute path to your `gcp_key_file`, and the two ports your Dockerfiles expose. The service names, the schedule, and the search payload are already filled in.

Both `gcloud_command.sh` and `test_hw3.sh` start by **sourcing** it:

```
SCRIPT_DIR=$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)
source "$SCRIPT_DIR/config.sh"
```

`source` runs a file in the *current* shell instead of a subshell, so every variable it assigns stays set in the script that sourced it. That is why you fill these values in once and both scripts see them — and why `config.sh` contains assignments only.

4. `gcloud_command.sh` (fill in `TODO 6–TODO 13`)

The script is written for you, with its five steps commented. Replace each `FILL_IN_<n>` token — nine of them, each with a numbered comment above it saying what belongs there. Most are the name of a variable `config.sh` already defines.

Do not paste literal project ids, regions, or ports into these commands. If you find yourself typing a value twice, it belongs in `config.sh`. Please remember that we will use the grader's `config.sh`.

What the five steps do, so the blanks make sense:

Step	What happens
1	Uploads your key to Secret Manager and grants Cloud Run permission to read it
2	<code>gcloud run deploy --source ./fastapi</code> — Cloud Build builds the Dockerfile, pushes and deploys the image
3	Reads the deployed API's public URL back out of Cloud Run
4	Deploys the dashboard, passing that URL in as an environment variable
5	Creates a Cloud Scheduler job that POSTs to the API once a day

Run it from this directory:

```
bash gcloud_command.sh
```

How to Verify

`test_hw3.sh` is your self-check (and for grading).

```
bash test_hw3.sh
```

It runs in three phases:

Phase	What it does
-------	--------------

Phase	What it does
1. Teardown	Deletes both Cloud Run services, the scheduler job, and every <code>job_search/*.json</code> file in your bucket
2. Local	Builds both Dockerfiles, runs both containers, POSTs a search, confirms a new file appears in the bucket and the dashboard
3. Cloud	Runs <code>gcloud_command.sh</code> , then checks the deployed API, dashboard, and scheduler job

Clean up when you are done. Leaving two Cloud Run services deployed burns free-tier quota, and the scheduler job will keep POSTing every day.

THINGS TO BE CAUTIONS

- **COPY * . fails in Cloud Build.** When `COPY`'s source can match more than one file, the **destination must be a directory ending in / in Cloud Build.**
- **The port in three places must agree:** your `EXPOSE`, your `CMD` flag, and `--port` in the deploy command.
- **Region has to match everywhere.** Cloud Run and Cloud Scheduler both take a location; the scheduler job must be in the same region you deployed to.

Submission Checklist

- ☐ `fastapi/Dockerfile` builds and serves on port 8000.
- ☐ `streamlit/Dockerfile` builds and serves on port 8501, bound to `0.0.0.0`.
- ☐ No credential, key file, or `.env` is copied into either image.
- ☐ `bash test_hw3.sh` completes with no `FAIL` lines.
- ☐ The deployed dashboard URL loads and renders the job table.
- ☐ You tore down your services after testing (`bash test_hw3.sh teardown`).
- ☐ Submit a zip called **HW3** containing **only 3 files:** `fastapi/Dockerfile`, `streamlit/Dockerfile`, and `gcloud_command.sh`.